

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Constraint Programming

Search II: Local Search, Restarts and Learning

Week 4

Search II: Local Search, Restarts and Learning

Assoc. Prof. Tolga Berber

KTU Faculty of Science, Department of Computer Science · https://avesis.ktu.edu.tr/tberber

Learning phases: 1 • Pre-instruction 2 • Content 3 • Participation 4 • Assessment 5 • Follow-through

1 • Pre-instruction Why It Matters?

Ask Week 3's backtracking search to place a million queens on a million-by-million board, and you could wait long past the end of this course for an answer – the search tree alone would not fit in memory.

- A completely different kind of search starts from a full (bad) guess and just keeps nudging it toward fewer conflicts, one queen at a time
- The classic literature result behind it finds a solution in about a minute – same problem, a fundamentally different way of searching
- By the end of this week you know four such search ideas, and why together they help explain why CP-SAT is fast

This Week

◎ Learning Objectives

By the end of this week, you will be able to:

1. Explain local search, state the min-conflicts heuristic, and describe how tabu search and simulated annealing escape a local minimum
2. Explain and apply large neighbourhood search: the destroy/ repair loop, choosing a neighbourhood, and using CP-SAT as the repair engine
3. Explain how restarts with nogood learning avoid repeating a discovered failure, and what lazy clause generation buys CP-SAT
4. Configure and run a portfolio/parallel search in CP-SAT, and explain what each worker contributes
5. Select and justify a combination of this week's techniques for an instance systematic search struggles with, and verify the real result

SECTION 1

Unit 1 — Local Search for CSPs

2 • Content Local Search: A Different Search Shape

- Week 3's search tree builds a partial assignment one variable at a time, guided by ordering heuristics
- Local search instead starts from a complete assignment – every variable already has a value, some constraints may be violated – and repeatedly changes one variable to reduce the violation count

Definition: Local search

A search method that starts from a complete (but possibly constraint-violating) assignment and repeatedly modifies it, one step at a time, to reduce the number of violated constraints.

Analogy: A Wedding Seating Chart, One Guest at a Time

Analogy

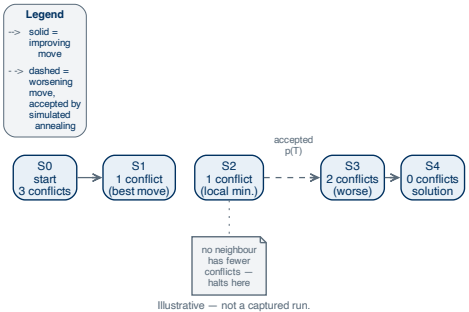
Instead of designing the whole seating chart from scratch, you start from a full (possibly bad) seating and repeatedly move the most awkwardly placed guest to the seat that causes the fewest conflicts, checking again, until – hopefully – no one is unhappy.

There is no tree here, and no way to prove no good seating exists – only “a seating was found” or “the time budget ran out.”

The Min-Conflicts Heuristic, and Two Ways to Escape

- **Min-conflicts:** pick a variable currently in a violated constraint, reassign it to the value that violates the fewest constraints, break ties at random
- **Local minimum:** a complete assignment where no single reassignment strictly helps – min-conflicts alone can get stuck here forever
- **Tabu search:** briefly forbids re-selecting a just-changed variable/value pair, stopping the search from undoing its own last move
- **Simulated annealing:** occasionally accepts a strictly *worse* move, with a probability that starts high and cools over the run

Escaping a Local Minimum



Min-conflicts' single greedy move stalls at a local minimum; simulated annealing's willingness to accept one temporarily worse move walks out of it. Illustrative labels, not a captured run.

Code: Setting Up the Occupancy Counters

A random complete assignment (fixed seed), plus row/diagonal occupancy counters for an O(1)-per-candidate conflict count:

```
rng = random.Random(seed)
columns = [rng.randrange(n) for _ in range(n)]
col_count = [0] * n
diag1_count = [0] * (2 * n - 1) # index: row - col + (n - 1)
diag2_count = [0] * (2 * n - 1) # index: row + col
for row, col in enumerate(columns):
    col_count[col] += 1
    diag1_count[row - col + n - 1] += 1
    diag2_count[row + col] += 1
```

Code: The Min-Conflicts Repair Step

```
col_count[columns[row]] -= 1
diag1_count[row - columns[row] + n - 1] -= 1
diag2_count[row + columns[row]] -= 1

best_columns, best_conflicts = [], n + 1
for col in range(n):
   conflicts = col_count[col] + diag1_count[row - col + n - 1] + diag2_count[row + col]
   if conflicts < best_conflicts:
      best_columns, best_conflicts = [col], conflicts
   elif conflicts == best_conflicts:
      best_columns.append(col)
new_col = rng.choice(best_columns)

columns[row] = new_col
col_count[new_col] += 1
diag1_count[row - new_col + n - 1] += 1
diag2_count[row + new_col] += 1
```

Every candidate column's conflict count comes straight from the counters – no O(N) recount per candidate.

Code and Output: Solving N=2000 Queens

```
for step in range(1, max_steps + 1):
   conflicted = _conflicted_rows(n, columns, col_count, diag1_count, diag2_count)
   if not conflicted:
      print(f"Solved N={n}: 0 conflicts remaining after {step - 1} steps.")
      return columns, step - 1
   row = rng.choice(conflicted)
   min_conflicts_step(row, n, columns, col_count, diag1_count, diag2_count, rng)
remaining = len(_conflicted_rows(n, columns, col_count, diag1_count, diag2_count))
print(f"Stopped N={n}: {remaining} conflicted rows remain after {max_steps} steps.")
```

Expected Output

```
Solved N=2000: 0 conflicts remaining after 1288 steps.
```

A real run, not the literature's million-queens claim – kept modest for a course lab.

3 • Participation Quick Check: Hand-Trace Min-Conflicts

Four queens, worst case: $Q_1 = 1, Q_2 = 2, Q_3 = 3, Q_4 = 4$ (every pair attacks every other pair). Q_2 is next to repair – for each column 1–4, count conflicts against the fixed $Q_1 = 1, Q_3 = 3, Q_4 = 4$.

Column 1: 1 conflict; column 2 (current): 3; column 3: 1; column 4: 2. Columns 1 and 3 tie for the minimum – min-conflicts moves Q_2 to whichever the tie-break rule picks.

SECTION 2

Unit 2 — Large Neighbourhood Search

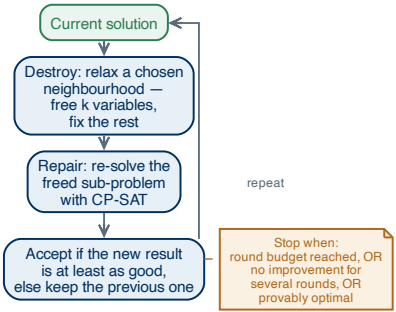
2 • Content From One Variable to a Whole Neighbourhood

- Min-conflicts repairs one variable at a time – fast, but shortsighted: sometimes only changing ten variables together escapes a bad neighbourhood
- Large neighbourhood search keeps the same habit – start from a solution, make it better – but hands each repair step to the full CP-SAT solver

Definition: Large neighbourhood search (LNS)

From an existing solution, relax (“destroy”) a chosen subset of variables while fixing the rest, then hand the freed sub-problem to a complete solver to re-optimize (“repair”) before deciding whether to keep the result.

The Destroy/Repair Loop



Schematic — shared LNS loop pattern.

Destroy a chosen neighbourhood, repair it with CP-SAT, accept only if the result is at least as good, and repeat.

Choosing a Neighbourhood

- A purely **random** subset of variables is the simplest destroy rule, and a reasonable default
- A **targeted** destroy rule – relax the variables actually involved in current violations, plus a few extra for search room – often converges faster
- LNS's repair step can find the best possible repair for that chunk, not just a locally-best single move

Analogy: Renovating One Room While Living in the Rest

Analogy

You do not tear down and rebuild the whole house (systematic search) or move one piece of furniture (local search) – you pick a whole room (a neighbourhood of variables), clear it out, and let a professional planner (the CP-SAT solver) work out the best layout for just that room while the rest of the house stays exactly as it was.

Code: Starting Point and Which Rows to Relax

```
rng = random.Random(seed)
columns = [rng.randrange(n) for _ in range(n)]
```

```
relaxed_rows = rng.sample(range(n), k)
```

A random start for $N = 50$ queens, then $k = 8$ rows chosen at random each round.

Code: Repairing the Relaxed Rows, the Constraints

```
model.AddAllDifferent([relaxed_vars[r] for r in relaxed_rows])
model.AddAllDifferent([relaxed_vars[r] + r for r in relaxed_rows])
model.AddAllDifferent([relaxed_vars[r] - r for r in relaxed_rows])
for row in relaxed_rows:
   for fixed_row in fixed_rows:
      fixed_col = columns[fixed_row]
      model.Add(relaxed_vars[row] != fixed_col)
      model.Add(relaxed_vars[row] + row != fixed_col + fixed_row)
      model.Add(relaxed_vars[row] - row != fixed_col - fixed_row)
```

`AllDifferent` among the relaxed rows, plus one clash constraint against every fixed row's column and both diagonals.

Code: Repairing the Relaxed Rows, Solve and Accept

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    return columns
proposal = list(columns)
for row in relaxed_rows:
    proposal[row] = solver.Value(relaxed_vars[row])
return proposal if count_conflicts(proposal) <= count_conflicts(columns) else columns
```

Keep the proposal only if conflicts did not increase.

Code and Output: The LNS Loop, a Real Conflict Trend

```python
for round_number in range(1, rounds + 1):
    relaxed_rows = lns_destroy(n, k, rng)
    columns = lns_repair(columns, relaxed_rows, n)
    conflicts = count_conflicts(columns)
    print(f"Round {round_number}: {conflicts} conflicts")
```

Expected Output

```
Round 0: 64 conflicts
Round 5: 7 conflicts
...
Round 29: 0 conflicts
Round 30: 0 conflicts
```

The Golomb Ruler: Marks in Increasing Order

Definition: Golomb ruler

A set of marks at distinct integer positions such that every pair of marks measures a different distance from every other pair; commonly posed as minimizing the total length for a fixed number of marks.

```
marks = [model.NewIntVar(0, upper_bound, f"mark_{i}") for i in range(
    order)]
model.Add(marks[0] == 0)
for i in range(order - 1):
    model.Add(marks[i] < marks[i + 1])
```

The Golomb Ruler: Every Pairwise Distance Distinct

```
distances = []
for i in range(len(marks)):
   for j in range(i + 1, len(marks)):
      distance = model.NewIntVar(1, upper_bound, f"dist_{i}_{j}")
      model.Add(distance == marks[j] - marks[i])
      distances.append(distance)
model.AddAllDifferent(distances)
```

One `AddAllDifferent` call over every pairwise mark difference – the ruler's entire "no two pairs share a gap" rule in one line.

Code and Output: Solving to Proven Optimality

```python
model.Minimize(marks[-1])
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
length = solver.Value(marks[-1])
positions = [solver.Value(mark) for mark in marks]
print(f"Status: {solver.StatusName(status)}")
print(f"Optimal length for {order} marks: {length}")
print(f"Marks: {positions}")
```

Expected Output

```
Status: OPTIMAL
Optimal length for 8 marks: 34
Marks: [0, 1, 4, 9, 15, 22, 32, 34]
```

Code and Output: Shrinking a Greedy Ruler

A greedy construction places each new mark at the smallest position keeping every distance seen so far distinct – length 44, not optimal.

```
for round_number in range(1, rounds + 1):
    relaxed = rng.sample(range(1, order), neighbourhood_size)
    proposal = _repair_relaxed_marks(marks, relaxed, order, upper_bound)
    if proposal is not None and _ruler_length(proposal) <= _ruler_length(marks):
        marks = proposal
    print(f"Round {round_number}: length {_ruler_length(marks)}, marks {marks}")
```

Expected Output

```
Round 0: length 44, marks [0, 1, 3, 7, 12, 20, 30, 44]
Round 3: length 41, marks [0, 1, 3, 7, 15, 20, 31, 41]
...
Round 15: length 41, marks [0, 1, 11, 13, 18, 32, 38, 41]
```

3 • Participation Quick Check: Choose a Neighbourhood

$N{=}50$ large-N-queens run: 6 conflicts remain, all involving rows 12, 13, 27 and 41. Which rows do you relax, which do you fix, and what must the repair step guarantee about the relaxed rows?

Relax at least rows 12, 13, 27, 41 (plus a few extra rows for search room); fix every other row's column as a constant. The repair sub-model must place the relaxed rows so none conflicts with each other or with any fixed row.

SECTION 3

Unit 3 — Restarts and Nogood Learning; Lazy Clause Generation

2 • Content Restarts Without Memory

- **Restart:** abandon the current search branch entirely and begin again from the root, typically with a different value order or random seed
- On its own – no memory kept between attempts – a restart can walk straight back into the exact same dead end it just escaped

CP-SAT never does this by accident: every failure becomes a permanent rule before the next attempt begins.

Nogood: A Rule Learned From a Failure

Branch $V_1 = a$, then $V_2 = b$; only then V_3 has no legal value left, no matter what V_4 is later set to – the failure came entirely from $V_1 = a$ and $V_2 = b$ together.

Definition: Nogood

A recorded combination of variable assignments proven, by a discovered failure, to never be extendable to a solution.

This toy justifies recording $\neg(V_1{=}a \wedge V_2{=}b)$.

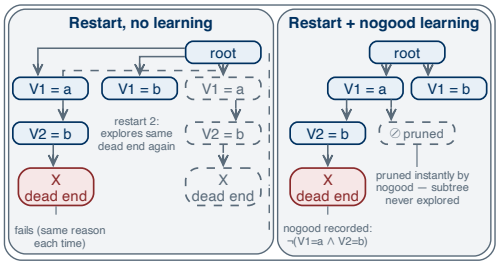
Analogy: A Note Pinned to a Shared Booking Calendar

Analogy

“Room 204 + Tuesday 9am + the microscope reservation = never works, already tried and failed” – pinned up so no one on the team wastes time proposing that exact combination again.

Nogood learning: record a nogood at every failure, and use the growing set to prune later parts of the *same* search, including after a restart.

Two Restarts, One With Memory



Same failure, discovered once — the right panel never pays for it twice.
Illustrative — schematic search trees (toy example), not a captured solver log.

Restart without learning repeats the same failing subtree; restart with a recorded nogood prunes the same failure instantly on its second occurrence.

Lazy Clause Generation: Why CP-SAT Is Fast

- Each constraint's own propagator can *explain*, on demand, exactly why it removed a value or caused a failure – as a clause over the decisions that caused it
- Those explanations feed a SAT-style clause-learning engine – learning from any propagator's failure, not only a hand-encoded SAT formula
- Restarts with nogood learning never repeat the same mistake twice; lazy clause generation is what lets that happen automatically, for every constraint CP-SAT supports

Code: Asking CP-SAT for a Verbose Log

```
solver = cp_model.CpSolver()
solver.parameters.log_search_progress = True
solver.parameters.max_time_in_seconds = time_limit
solver.parameters.random_seed = seed
solver.parameters.num_search_workers = 1
status = solver.Solve(model)
print(f"Status: {solver.StatusName(status)}")
print(f"Length for {order} marks: {solver.Value(marks[-1])}")
```

Output: A Real Restart/Conflict Log

Expected Output

```
CpSolverResponse summary:
status: OPTIMAL
objective: 44
conflicts: 8573
restarts: 1139
walltime: 0.169559
```

9-mark Golomb-ruler proof, single worker – every restart and conflict here belongs to one systematic search.

3 • Participation Quick Check: Write the Nogood

Backtracking on a 4-variable CSP: branch $V_1 = a$, then $V_2 = b$, then discover V_3 has no legal value left regardless of V_4. Write the nogood, and say which branch it prevents on the next restart.

The nogood is $\neg(V_1{=}a \wedge V_2{=}b)$. On the next restart (or any later branch), trying $V_1 = a$ followed by $V_2 = b$ fails immediately – the subtree that originally revealed the failure is never re-explored.

SECTION 4

Unit 4 — Portfolios and Parallel Search

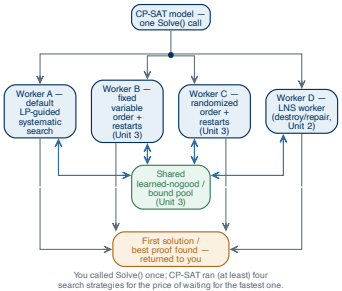
2 • Content Several Strategies at Once

- So far this week: one strategy at a time – local search, then LNS, then restarts with learning
- Set one CP-SAT parameter, and it runs several of these at once, on separate cores, keeping whichever result comes back first

Definition: Portfolio search

Running several different search strategies concurrently on the same problem instance, and keeping the result of whichever finishes first (or proves the tightest bound).

What `num_search_workers` Actually Launches



Roughly k different concurrent workers – systematic variants with different orders and restart policies, and dedicated LNS workers – sharing learned nogoods and bounds while running.

Practical Guidance, and an Honest Limit on the Speed-Up

- A sensible start: one worker per available core – returns diminish well before that
- Not every worker's job is to speed up *the specific answer you are waiting for* – an LNS worker chases a better solution, not necessarily a faster optimality proof
- Coordinating shared information (nogoods, bounds) has its own cost – adding workers is not a clean multiplicative speed-up
- `random_seed` fixes a single-worker run deterministically; a multi-worker run is not guaranteed to reproduce the same timing, since workers' relative progress depends on real wall-clock scheduling

Analogy: Several Travel Agents, One Trip

Analogy

You ask several travel agents to plan the same trip at once, each using a different strategy – cheapest fare first, fewest layovers first, fastest arrival first. Whichever comes back with a usable itinerary first, you take it – you never needed to know in advance which strategy would win.

Code: Single- vs Multi-Worker Runs

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = 1
solver.parameters.random_seed = seed
start = time.perf_counter()
status = solver.Solve(model)
wall_time = time.perf_counter() - start
```

```
solver = cp_model.CpSolver()
solver.parameters.num_search_workers = workers
solver.parameters.random_seed = seed
start = time.perf_counter()
status = solver.Solve(model)
wall_time = time.perf_counter() - start
```

Same 9-mark Golomb-ruler proof, same fixed seed – only `num_search_workers` changes between the two calls.

Output and Figure: The Real Speed-Up

Expected Output

```
num_search_workers=1  wall_time=0.1707s  length=44  status=OPTIMAL
num_search_workers=8  wall_time=0.0443s  length=44  status=OPTIMAL
speed-up (1-worker time / 8-worker time): 3.86x
```



3 • Participation Quick Check: Is 8 Workers 8× Faster?

Using the real numbers above, is the 8-worker speed-up close to $8\times$, or clearly less? Give two reasons from this unit.

Clearly less – $3.86\times$ on this run, not $8\times$. (1) not every worker's task shortens the specific proof you are waiting for; (2) workers share information and different workers suit different instances, so added workers give diminishing, not multiplicative, returns.

SECTION 5

Summary

Summary

Summary

- Local search repairs a complete assignment step by step – fast, but it can never prove infeasibility
- LNS generalizes “repair one variable” to “repair a whole relaxed neighbourhood with CP-SAT”
- Nogood learning turns every search failure into a permanent rule – restarts stop repeating the same mistake
- Portfolio search runs several of these strategies at once, at a real but clearly sub-linear speed-up

4 • Assessment Check Yourself

- One Check Yourself question closes each unit in this week's chapter – you will find the answers collected at the chapter's end
- Why min-conflicts can get permanently stuck and how simulated annealing escapes it, one LNS iteration step by step, why nogood-recording converges faster, why 8 workers is not $8\times$ faster

5 • Follow-through Transfer — Where Else?

- Weeks 12 and 14 lean on this week's local-search and LNS family whenever a problem is too large for systematic search to handle from scratch (rostering, routing)
- Every "CP-SAT versus your own Week 3 backtracker" comparison for the rest of this course favours CP-SAT largely because of nogood learning, not raw speed
- Week 15's infeasibility work formalizes "explain a failure" for a human, not just for pruning a search
- This course has no running capstone project – the thread is the *same* classic problems, revisited with new machinery

In the Lab

This week's lab sheet

You run min-conflicts and watch its conflict count fall to zero, build and run the large-N-queens and Golomb-ruler LNS loops, locate real restart/conflict lines in a captured solver log, and compare real single- versus multi-worker wall-clock times.

- Capstone step: given a harder instance than any single script above solves comfortably, choose and justify a combination of at least two of this week's techniques
- Next week: Week 5 asks why CP-SAT is fast in the worst case too – treewidth and symmetry breaking